САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МЕХАНИКИ И ОПТИКИ
ФАКУЛЬТЕТ ИНФОКОММУНИКАЦИОННЫХ ТЕХНОЛОГИЙ

Отчет по лабораторной работе №4
по курсу «Проектирование и реализация баз данных»
Вариант 10 “Vokzal”

Выполнил:
Гашимов И.Ф.
К3239
Проверила:
Говорова М.М.

Санкт-Петербург

2026 г.

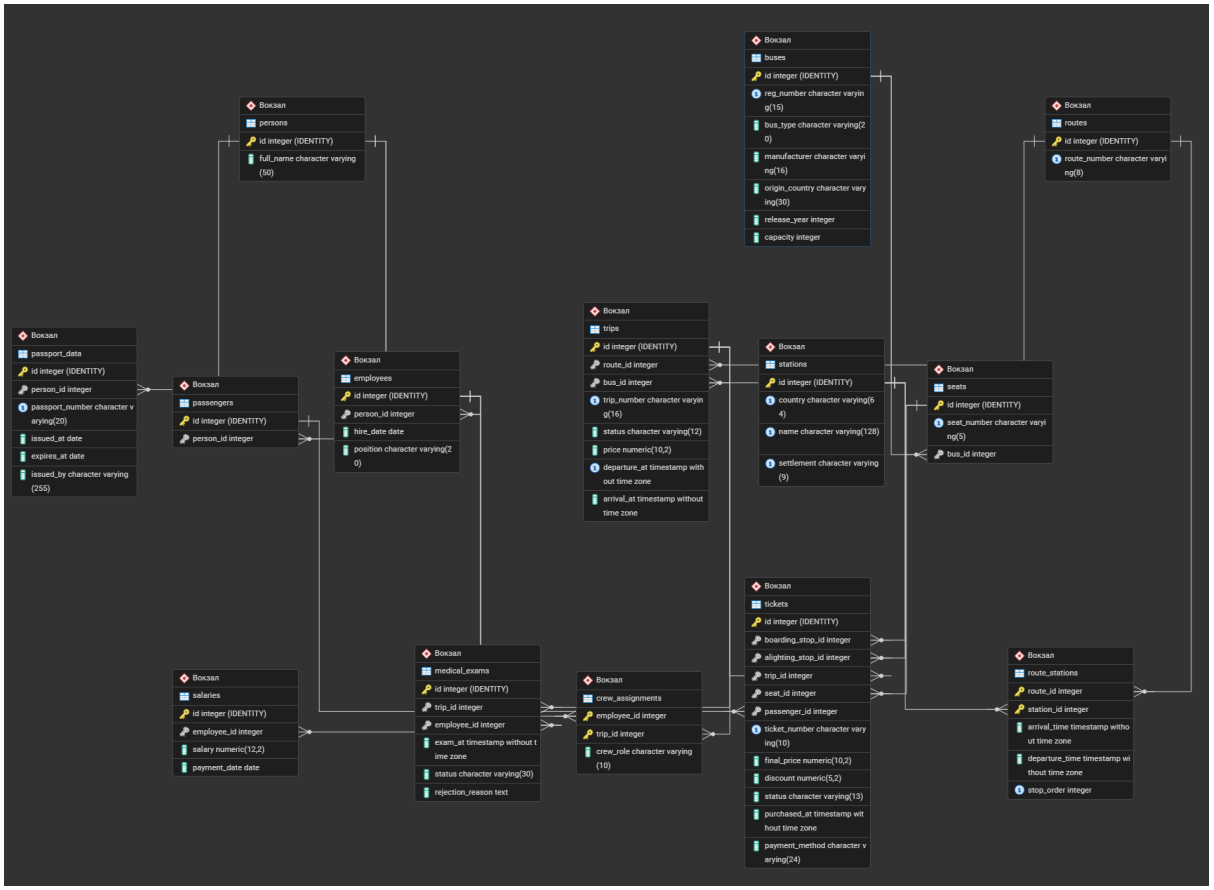
Цель работы: овладеть практическими навыками создания представлений и запросов на выборку данных к базе данных PostgreSQL, использования подзапросов при модификации данных и индексов.

Практическое задание:

- создать запросы и представления на выборку данных согласно индивидуальному заданию;
- составить 3 запроса на модификацию данных (INSERT, UPDATE, DELETE) с подзапросами;
- изучить графическое представление запросов и историю запросов;
- создать простой и составной индексы, сравнить время выполнения запросов без индексов и с индексами, использовать EXPLAIN.

Схема базы данных

В БД созданы таблицы: route, stop, stop_in_route,
trip, ticket, bus, seat, passenger, medical_exam, employee, salary,
person, passport_data, assigned_to_crew.



ERD в нативном качестве есть в репозитории 3 работы

1. Выполнение работы

1.1 Запросы на выборку данных

- Вывести фамилии водителей и номера автобусов, отправившиеся в рейсы до 12 часов текущего дня.
- Рассчитать выручку от продажи билетов за прошедший день.
- Вывести список водителей, которые не выполнили ни одного рейса за прошедшую неделю.
- Вывести сумму убытков из-за непроданных мест в автобусе за прошедшую неделю.
- Найти самый популярный маршрут за прошедший месяц.
- Вывести тип автобуса, который используется на всех рейсах.
- Вывести данные водителя, который провел максимальное время в пути за прошедшую неделю.

1 Запрос: Вывести фамилии водителей и номера автобусов, отправившиеся в рейсы до 12 часов текущего дня.

```
SELECT DISTINCT
   fl."full_name" AS "Driver",
   a."number" AS "bus_number",
   r."number" AS "trip_number",
   r."departure_date" AS "departure"
FROM "Vokzal"."trip" r
JOIN "Vokzal"."bus" a
   ON a."bus_id" = r."bus_id"
JOIN "Vokzal"."assigned_to_crew" ne
   ON ne."trip_id" = r."trip_id"
JOIN "Vokzal"."employee" s
   ON s."employee_id" = ne."employee_id"
JOIN "Vokzal"."person" fl
   ON fl."person_id" = s."person_id"
WHERE s."position" = 'Водитель'
  AND r."status" IN ('Отправлен', 'Прибыл')
  AND r."departure_date" >= CURRENT_DATE
  AND r."departure_date" < CURRENT_DATE + INTERVAL '12 hours'
ORDER BY r."departure_date", fl."full_name";
```

Запрос выбирает рейсы, которые отправились сегодня до 12:00. Далее через таблицу назначения в экипаж определяется, какие сотрудники были назначены на рейс, и оставляются только сотрудники с должностью «Водитель». В результате выводятся ФИО водителя, номер автобуса, номер рейса и время отправления.

2 Запрос: Рассчитать выручку от продажи билетов за прошедший день.

```
SELECT
   COALESCE(SUM(b."price"), 0) AS "revenue_yesterday"
FROM "Vokzal"."ticket" b
WHERE b."purchase_date" >= CURRENT_DATE - INTERVAL '1 day'
  AND b."purchase_date" < CURRENT_DATE
  AND b."status" IN ('Куплен', 'Использован');
```

Запрос суммирует стоимость всех билетов, купленных за предыдущий день. Учитываются только билеты со статусами «Куплен» и «Использован», чтобы исключить отмененные или недействительные продажи.

3 Запрос: Вывести список водителей, которые не выполнили ни одного рейса за прошедшую неделю.

```
SELECT
   fl."full_name" AS "Driver"
FROM "Vokzal"."employee" s
JOIN "Vokzal"."person" fl
   ON fl."person_id" = s."person_id"
WHERE s."position" = 'Водитель'
  AND NOT EXISTS (
     SELECT 1
     FROM "Vokzal"."assigned_to_crew" ne
     JOIN "Vokzal"."trip" r
        ON r."trip_id" = ne."trip_id"
     WHERE ne."employee_id" = s."employee_id"
       AND r."status" IN ('Отправлен', 'Прибыл')
       AND r."departure_date" >= CURRENT_DATE - INTERVAL '7 days'
       AND r."departure_date" < CURRENT_DATE

ORDER BY fl."full_name";
```

Запрос находит всех сотрудников с должностью «Водитель» и проверяет, были ли у них рейсы за последние 7 дней. Если у водителя не найдено ни одного отправленного или завершенного рейса, он попадает в результат.

4 Запрос: Вывести сумму убытков из-за непроданных мест в автобусе за прошедшую неделю.

```
WITH sold AS (
   SELECT
      b."trip_id",
      COUNT(*) AS sold_seats
   FROM "Vokzal"."ticket" b
   WHERE b."seat_id" IS NOT NULL
```

```
    AND b."status" IN ('Куплен', 'Использован')
  GROUP BY b."trip_id"
)
SELECT
  COALESCE(
    SUM(
      GREATEST(a."seat_count" - COALESCE(s.sold_seats, 0), 0)
      * COALESCE(r."price", 0)
    ),
    0
  )::numeric(12,2) AS "losses_due_to_unsold_seats"
FROM "Vokzal"."trip" r
JOIN "Vokzal"."bus" a
  ON a."bus_id" = r."bus_id"
LEFT JOIN sold s
  ON s."trip_id" = r."trip_id"
WHERE r."status" IN ('Отправлен', 'Прибыл')
  AND r."departure_date" >= CURRENT_DATE - INTERVAL '7 days'
  AND r."departure_date" < CURRENT_DATE;
```

Сначала подсчитывается количество проданных мест по каждому рейсу. Затем для каждого рейса определяется число непроданных мест как разность между вместимостью автобуса и числом проданных билетов. После этого количество непроданных мест умножается на цену рейса, и полученные значения суммируются за последнюю неделю.

5 Запрос: Найти самый популярный маршрут за прошедший месяц.

```
SELECT
  m."route_number" AS "route_number",
  COUNT(r."trip_id") AS "trip_count"
FROM "Vokzal"."trip" r
JOIN "Vokzal"."route" m
  ON m."route_id" = r."route_id"
WHERE r."status" IN ('Отправлен', 'Прибыл')
  AND r."departure_date" >= CURRENT_DATE - INTERVAL '1 month'
  AND r."departure_date" < CURRENT_DATE
GROUP BY m."route_id", m."route_number"
HAVING COUNT(r."trip_id") = (
  SELECT MAX(route_count)
  FROM (
    SELECT COUNT(r2."trip_id") AS route_count
    FROM "Vokzal"."trip" r2
    WHERE r2."status" IN ('Отправлен', 'Прибыл')
      AND r2."departure_date" >= CURRENT_DATE - INTERVAL '1 month'
      AND r2."departure_date" < CURRENT_DATE
    GROUP BY r2."route_id"
  ) t
);
```

Запрос подсчитывает количество выполненных рейсов по каждому маршруту за последний месяц. Затем выбираются все маршруты, у которых это количество равно максимальному значению. Поэтому при одинаковом числе рейсов будут выведены все такие маршруты.

6 Запрос: Вывести тип автобуса, который используется на всех рейсах. (Возможно имелось ввиду Тип автобуса, который используется на всех маршрутах. То есть для каждого маршрута есть хотя бы один рейс этого типа.)

```
SELECT
a."type" AS "bus_type"
FROM "Vokzal"."trip" r
JOIN "Vokzal"."bus" a
ON a."bus_id" = r."bus_id"
GROUP BY a."type"
HAVING COUNT(DISTINCT r."route_id") = (
SELECT COUNT(*)
FROM "Vokzal"."route"
)
ORDER BY a."type";
```

Запрос определяет, на скольких различных маршрутах использовался каждый тип автобуса. Если количество маршрутов для данного типа совпадает с общим количеством маршрутов в базе данных, значит этот тип автобуса используется на всех маршрутах.

7 Запрос: Вывести данные водителя, который провел максимальное время в пути за прошедшую неделю.

```
WITH driver_time AS (
  SELECT
    s."employee_id",
    fl."full_name",
    SUM(r."arrival_date" - r."departure_date") AS "total_travel_time"
  FROM "Vokzal"."assigned_to_crew" ne
  JOIN "Vokzal"."employee" s
    ON s."employee_id" = ne."employee_id"
  JOIN "Vokzal"."person" fl
    ON fl."person_id" = s."person_id"
  JOIN "Vokzal"."trip" r
    ON r."trip_id" = ne."trip_id"
  WHERE s."position" = 'Водитель'
   AND r."status" IN ('Отправлен', 'Прибыл')
   AND r."departure_date" >= CURRENT_DATE - INTERVAL '7 days'
   AND r."departure_date" < CURRENT_DATE
```

```
    GROUP BY s."employee_id", fl."full_name"
)
SELECT
    dt."full_name" AS "Driver",
    pd."number" AS "passport",
    dt."total_travel_time"
FROM driver_time dt
JOIN "Vokzal"."employee" s
    ON s."employee_id" = dt."employee_id"
LEFT JOIN "Vokzal"."passport_data" pd
    ON pd."person_id" = s."person_id"
WHERE dt."total_travel_time" = (
    SELECT MAX("total_travel_time")
    FROM driver_time
);
```

1.2. Создание представлений

1 Представление: Количество свободных мест на все рейсы на завтра.

```
CREATE OR REPLACE VIEW "Vokzal"."vw_free_seats_for_tomorrow" AS
SELECT
    r."trip_id",
    r."number" AS "trip_number",
    m."route_number" AS "route_number",
    r."departure_date" AS "departure",
    a."number" AS "bus_number",
    a."type" AS "bus_type",
    a."seat_count" AS "total_seats",
    COUNT(b."ticket_id") AS "occupied_seats",
    a."seat_count" - COUNT(b."ticket_id") AS "free_seats"
FROM "Vokzal"."trip" r
JOIN "Vokzal"."bus" a
    ON a."bus_id" = r."bus_id"
JOIN "Vokzal"."route" m
    ON m."route_id" = r."route_id"
LEFT JOIN "Vokzal"."ticket" b
    ON b."trip_id" = r."trip_id"
   AND b."seat_id" IS NOT NULL
   AND b."status" IN ('Куплен', 'Использован')
WHERE r."departure_date" >= CURRENT_DATE + INTERVAL '1 day'
  AND r."departure_date" < CURRENT_DATE + INTERVAL '2 days'
  AND r."status" <> 'Отменён'
GROUP BY
    r."trip_id",
    r."number",
    m."route_number",
    r."departure_date",
    a."number",
    a."type",
    a."seat_count";
```

Видим такой вывод:

Показаны строки: от 1 до 80 Номер страницы: 1 из 1

	ид_рейса integer	Номер_рейса character varying (16)	Номер_маршрута character varying (8)	Отправление timestamp without time zone	Номер_автобуса character varying (15)	Тип_автобуса character varying (20)	Всего_мест integer	Занято_мест bigint	Свободно_мест bigint
1	53	REIS-1053	936B	2025-12-11 15:45:00	ZA195BB47	Туристический	39	21	18
2	48	REIS-1048	940A	2025-12-12 12:00:00	SJ187BB41	Междугородний	55	13	42
3	43	REIS-1043	263M	2025-12-19 12:15:00	SJ187BB41	Междугородний	55	23	32
4	49	REIS-1049	979B	2025-12-22 17:30:00	HL283BB48	Междугородний	31	18	13
5	54	REIS-1054	371	2025-12-31 13:15:00	DC510BB72	Междугородний	55	20	35
6	47	REIS-1047	371	2025-12-31 16:30:00	HX516BB72	Туристический	44	11	33
7	44	REIS-1044	993	2026-01-03 11:45:00	HL283BB48	Междугородний	31	16	15

Представление показывает рейсы на завтра и количество свободных мест по каждому из них. Свободные места определяются как разность между вместимостью автобуса и числом уже занятых мест по проданным билетам.

2 Представление: Самый популярный маршрут этой зимой.

```
CREATE OR REPLACE VIEW
"Vokzal"."vw_most_popular_route_this_winter" AS
SELECT
   m."route_number" AS "route_number",
   COUNT(r."trip_id") AS "trip_count"
FROM "Vokzal"."trip" r
JOIN "Vokzal"."route" m
   ON m."route_id" = r."route_id"
WHERE r."status" IN ('Отправлен', 'Прибыл')
  AND EXTRACT(MONTH FROM r."departure_date") IN (12, 1, 2)
GROUP BY m."route_number"
HAVING COUNT(r."trip_id") = (
   SELECT MAX(route_count)
   FROM (
      SELECT COUNT(r2."trip_id") AS route_count
      FROM "Vokzal"."trip" r2
      WHERE r2."status" IN ('Отправлен', 'Прибыл')
        AND EXTRACT(MONTH FROM r2."departure_date") IN (12, 1, 2)
      GROUP BY r2."route_id"
   ) t
);
```

Представление определяет количество выполненных рейсов по каждому маршруту зимой. Затем выбираются все маршруты, у которых это количество максимально. Если несколько маршрутов имеют одинаковое максимальное значение, они будут выведены все.

Видим такой вывод:

Результат Сообщения Уведомления

Показаны строки: от 1 до 8

	Номер_маршрута character varying (8)	Количество_рейсов bigint
1	371	3
2	263M	2
3	940A	2
4	993	2
5	936B	1
6	979B	1
7	508Б	1
8	775A	1

1.3 Запросы на модификацию данных

Запрос INSERT с подзапросом:

```
INSERT INTO "Vokzal"."salary" ("employee_id", "salary_amount", "payment_date")
SELECT
   s."employee_id",
   COALESCE(
      (
         SELECT z."salary_amount"
         FROM "Vokzal"."salary" z
         WHERE z."employee_id" = s."employee_id"
         ORDER BY z."payment_date" DESC
         LIMIT 1
      ),
      60000
   ) AS "salary_amount",
   CURRENT_DATE
FROM "Vokzal"."employee" s
WHERE s."employee_id" = (
   SELECT ne."employee_id"
   FROM "Vokzal"."assigned_to_crew" ne
   GROUP BY ne."employee_id"
   ORDER BY COUNT(*) DESC, ne."employee_id"
   LIMIT 1
);
```

Запрос добавляет новую запись о зарплате сотруднику, который чаще всего участвовал в рейсах. Размер оклада берется из его последней записи о зарплате, а если таких записей нет, подставляется значение 60000.

Ниже видим что запись действительно появилась:

Результат Сообщения Уведомления

Показаны строки: от 1 до 3 Номер страницы: 1 из 1

	ид_зарплаты [PK] integer	ид_сотрудника integer	оклад numeric (12,2)	дата_выплаты date
1	49	4	69170.00	2026-04-12
2	8	4	69170.00	2026-03-13
3	7	4	51315.00	2026-02-11

Запрос UPDATE с подзапросом:

Сначала выберем билет со статусом “Куплен” для последнего завершенного рейса. Покажем запись до изменения:

Показаны строки: от 1 до 1 Номер страницы: 1 из 1

	ид_билета [PK] integer	ид_остановки_посадки integer	ид_остановки_высадки integer	ид_рейса integer	ид_места integer	ид_пассажира integer	номер character varying (10)	цена numeric (10,2)	скидка numeric (5,2)	статус character varying (13)	дата_п timesta
1	655	38	7	35	333	20	T609529	3381.00	0.00	Куплен	2026-0

```
UPDATE "Vokzal"."ticket" b
SET "status" = 'Использован'
WHERE b."ticket_id" = (
   SELECT b2."ticket_id"
   FROM "Vokzal"."ticket" b2
   WHERE b2."status" = 'Куплен'
     AND b2."trip_id" = (
        SELECT r."trip_id"
        FROM "Vokzal"."trip" r
        WHERE r."arrival_date" < NOW()
          AND r."status" IN ('Отправлен', 'Прибыл')
        ORDER BY r."arrival_date" DESC
        LIMIT 1
     )
   ORDER BY b2."purchase_date" DESC
   LIMIT 1
);
```

Запрос изменяет статус выбранного билета с «Куплен» на «Использован». Подзапрос определяет нужный билет автоматически.

После выполнения видим:

Результат Сообщения Уведомления

Показаны строки: от 1 до 1 Номер страницы: 1 из 1

	ид_билета [PK] integer	ид_остановки_посадки integer	ид_остановки_высадки integer	ид_рейса integer	ид_места integer	ид_пассажира integer	номер character varying (10)	цена numeric (10,2)	скидка numeric (5,2)	статус character varying (13)	дата_по timestar
1	656	38	16	35	359	51	T397252	3381.00	0.00	Использован	2026-04

Запрос DELETE с подзапросом:

Сначала выбирается последняя запись о зарплате водителя, который участвовал в минимальном количестве рейсов. Видим до удаления:

SQL

	ид_зарплаты [PK] integer	ид_сотрудника integer	оклад numeric (12,2)	дата_выплаты date
1	14	7	44255.00	2026-03-13

```
DELETE FROM "Vokzal"."salary" z
WHERE z."salary_id" = (
   SELECT zp."salary_id"
   FROM "Vokzal"."salary" zp
   WHERE zp."employee_id" = (
      SELECT s."employee_id"
      FROM "Vokzal"."employee" s
      LEFT JOIN "Vokzal"."assigned_to_crew" ne
         ON ne."employee_id" = s."employee_id"
      WHERE s."position" = 'Водитель'
      GROUP BY s."employee_id"
      ORDER BY COUNT(ne."trip_id") ASC, s."employee_id"
      LIMIT 1
   )
   ORDER BY zp."payment_date" DESC, zp."salary_id" DESC
   LIMIT 1
);
```

Запрос удаляет последнюю запись о зарплате у водителя с минимальным числом рейсов. Нужная запись выбирается через подзапрос.

1.4 Работа с индексами и планами запросов

1.4.1 Простой индекс

вводим этот запрос без индекса:

```
SELECT
   COALESCE(SUM(b."price"), 0) AS "revenue_yesterday"
FROM "Vokzal"."ticket" b
WHERE b."purchase_date" >= CURRENT_DATE - INTERVAL '1 day'
  AND b."purchase_date" < CURRENT_DATE
  AND b."status" IN ('Куплен', 'Использован');
```

Explain:

Explain Analyze:

Graphical | Analysis | Statistics

#	Node	Строки Actual	Loops
1.	→ Aggregate (rows=1 loops=1)	1	1
2.	→ Seq Scan on Билет as b (rows=35 loops=1) **Фильтр**: ((("статус")::text = ANY ('{Куплен,Использован}'::text[])) AND ("дата_покупки" < CURRENT_DATE) AND ("дата_по **Строки, удаленные фильтром**: 1449	35	1

Видим время выполнения:

Результат | Сообщения | План выполнения × | Уведомления

```
Запрос выполнен успешно. Общее время выполнения: 53 мс.
обработано строк: 1.
```

Создадим простой индекс

```
CREATE INDEX "idx_ticket_purchase_date"
ON "Vokzal"."ticket" ("purchase_date");
```

Результат | Сообщения | План выполнения × | Уведомления

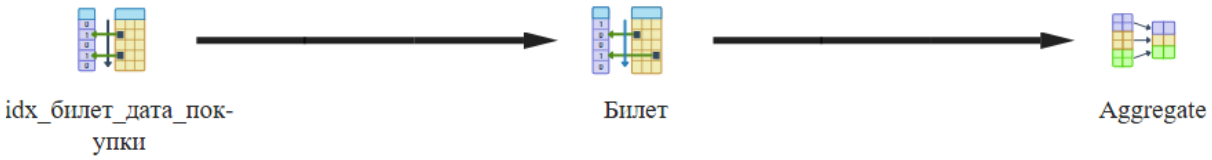
```
CREATE INDEX

Запрос завершён успешно, время выполнения: 53 мс.
```

Создан простой индекс по полю purchase_date, так как в запросе отбор записей выполняется именно по этой дате.

Выполняем повторно запрос с индексом:

```
SELECT
   COALESCE(SUM(b."price"), 0) AS "revenue_yesterday"
FROM "Vokzal"."ticket" b
WHERE b."purchase_date" >= CURRENT_DATE - INTERVAL '1 day'
  AND b."purchase_date" < CURRENT_DATE
  AND b."status" IN ('Куплен', 'Использован');
```

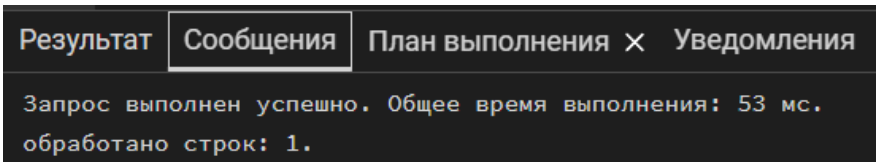
Explain:



Explain Analyze:

#	Node	Строки Actual	Loops
1.	→ Aggregate (rows=1 loops=1)	1	1
2.	→ Bitmap Heap Scan on Билет as b (rows=35 loops=1) **Фильтр**: (("статус")::text = ANY ("{Куплен,Использован}"::text[])) **Строки, удаленные фильтром**: 2 **Перепровеить условие**: (("дата_покупки" >= (CURRENT_DATE - '1 day'::interval)) AND ("дата_покупки" < CURRENT_DATE **Блоки кучи**: exact=5	35	1
3.	→ Bitmap Index Scan using idx_билет_дата_покупки (rows=37 loops=1) **Индекс состояния**: (("дата_покупки" >= (CURRENT_DATE - '1 day'::interval)) AND ("дата_покупки" < CURRENT_DATE	37	1

Видим время выполнения:



После создания индекса запрос выполняется повторно для сравнения плана выполнения и времени работы.

Вариант	Время выполнения
Без индекса	53 ms
С индексом	53 ms

В нашем случае индекс играет небольшую роль т.к. у нас не так много строк.

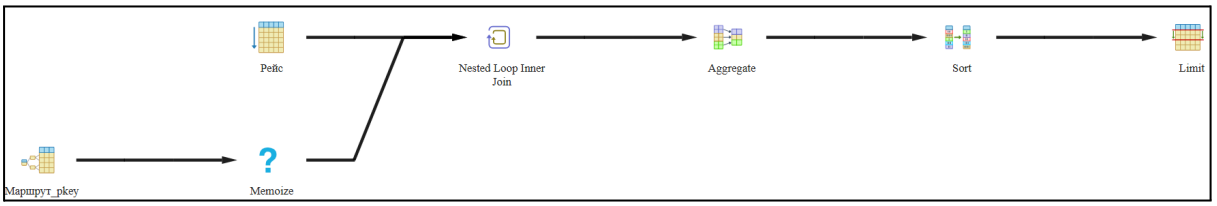
1.4.2 Составной индекс

Теперь делаем то же самое, но для второго запроса.
Возьмём запрос о самом популярном маршруте за прошедший месяц.

Выполняем запрос без индекса:

```
SELECT
   m."route_number" AS "route_number",
   COUNT(r."trip_id") AS "trip_count"
FROM "Vokzal"."trip" r
JOIN "Vokzal"."route" m
   ON m."route_id" = r."route_id"
WHERE r."status" IN ('Отправлен', 'Прибыл')
  AND r."departure_date" >= CURRENT_DATE - INTERVAL '1 month'
  AND r."departure_date" < CURRENT_DATE
GROUP BY m."route_id", m."route_number"
HAVING COUNT(r."trip_id") = (
   SELECT MAX(route_count)
   FROM (
      SELECT COUNT(r2."trip_id") AS route_count
      FROM "Vokzal"."trip" r2
      WHERE r2."status" IN ('Отправлен', 'Прибыл')
        AND r2."departure_date" >= CURRENT_DATE - INTERVAL '1 month'
        AND r2."departure_date" < CURRENT_DATE
      GROUP BY r2."route_id"
   ) t
);
```

Explain:



Explain Analyze:

Результат Сообщения План выполнения × Уведомления

Graphical Analysis Statistics

1.	→ Limit (rows=1 loops=1)	1	1
2.	→ Sort (rows=1 loops=1)	1	1
3.	→ Aggregate (rows=20 loops=1) **Buckets**: **Batches**: **Использование памяти**: 24 kB	20	1
4.	→ Nested Loop Inner Join (rows=47 loops=1)	47	1
5.	→ Seq Scan on Рейс as r (rows=47 loops=1) **Фильтр**: ((("статус")::text = ANY ('{Отправлен,Прибыл}'::text[])) AND ("дата_отправления" < CURRENT_D **Строки, удаленные фильтром**: 33	47	1
6.	→ Memoize (rows=1 loops=47) **Buckets**: **Batches**: **Использование памяти**: 3 kB	1	47
7.	→ Index Scan using Маршрут_pkey on Маршрут as m (rows=1 loops=20) **Индекс состояния**: ("ид_маршрута" = r."ид_маршрута")	1	20

Время выполнения:

Результат Сообщения План выполнения × Уведомления

```
Запрос выполнен успешно. Общее время выполнения: 61 мс.
обработано строк: 1.
```

Запрос определяет маршрут, по которому за последний месяц было выполнено максимальное количество рейсов. Сначала выполняется вариант без индекса.

Создадим составной индекс:

```
CREATE INDEX "idx_trip_departure_date_route"
ON "Vokzal"."trip" ("departure_date", "route_id");
```

Результат Сообщения План выполнения × Уведомления

```
CREATE INDEX

Запрос завершён успешно, время выполнения: 53 мс.
```

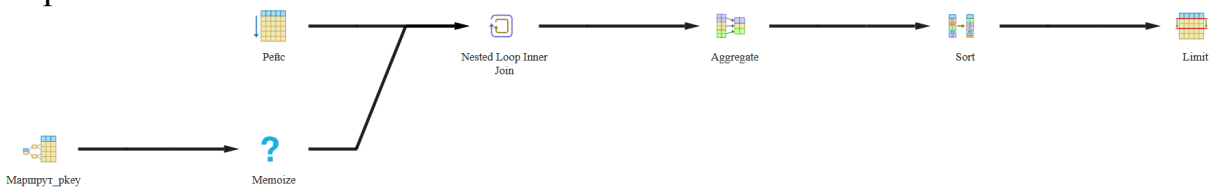
Создан составной индекс по полям departure_date и route_id, так как в запросе отбор выполняется по дате, а группировка связана с маршрутом.

Выполняем повторно запрос с индексом:

```
SELECT
```

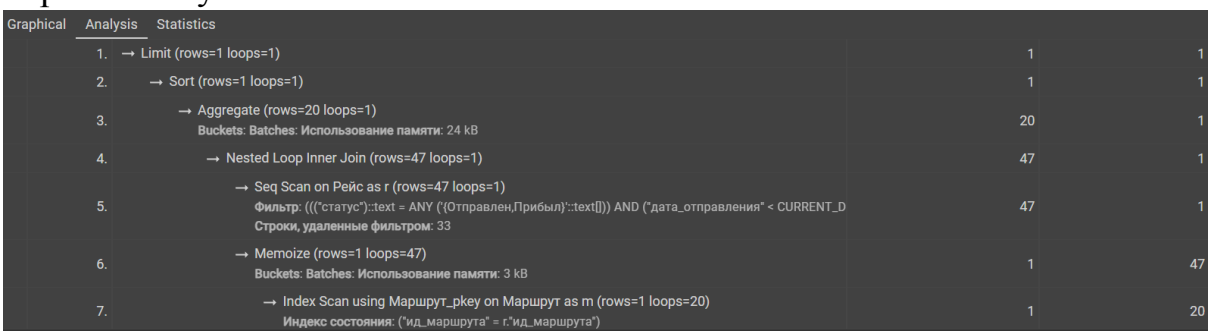
```
    m."route_number" AS "route_number",
    COUNT(r."trip_id") AS "trip_count"
FROM "Vokzal"."trip" r
JOIN "Vokzal"."route" m
    ON m."route_id" = r."route_id"
WHERE r."status" IN ('Отправлен', 'Прибыл')
  AND r."departure_date" >= CURRENT_DATE - INTERVAL '1 month'
  AND r."departure_date" < CURRENT_DATE
GROUP BY m."route_id", m."route_number"
HAVING COUNT(r."trip_id") = (
    SELECT MAX(route_count)
    FROM (
        SELECT COUNT(r2."trip_id") AS route_count
        FROM "Vokzal"."trip" r2
        WHERE r2."status" IN ('Отправлен', 'Прибыл')
          AND r2."departure_date" >= CURRENT_DATE - INTERVAL '1 month'
          AND r2."departure_date" < CURRENT_DATE
        GROUP BY r2."route_id"
    ) t
);
```

Explain:



Explain скорее всего не изменился так как сработали оптимизаторы и система решила, что и без этого будет быстро работать, но во времени разница будет.

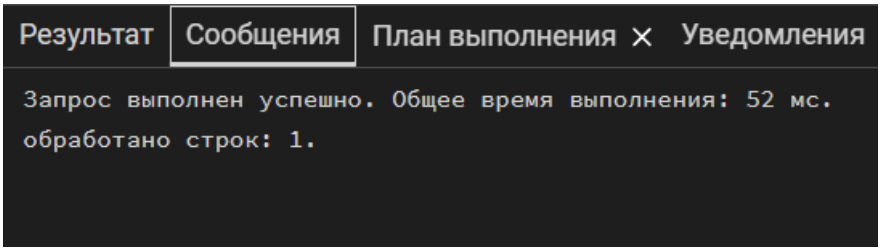
Explain Analyze:



Graphical | Analysis | Statistics

1.	→ Limit (rows=1 loops=1)	1	1
2.	→ Sort (rows=1 loops=1)	1	1
3.	→ Aggregate (rows=20 loops=1) **Buckets: Batches: Использование памяти**: 24 kB	20	1
4.	→ Nested Loop Inner Join (rows=47 loops=1)	47	1
5.	→ Seq Scan on Рейс as r (rows=47 loops=1) **Фильтр**: ((("статус")::text = ANY ('{Отправлен,Прибыл}'::text[])) AND ("дата_отправления" < CURRENT_D **Строки, удаленные фильтром**: 33	47	1
6.	→ Memoize (rows=1 loops=47) **Buckets: Batches: Использование памяти**: 3 kB	1	47
7.	→ Index Scan using Маршрут_pkey on Маршрут as m (rows=1 loops=20) **Индекс состояния**: ("ид_маршрута" = r."ид_маршрута")	1	20

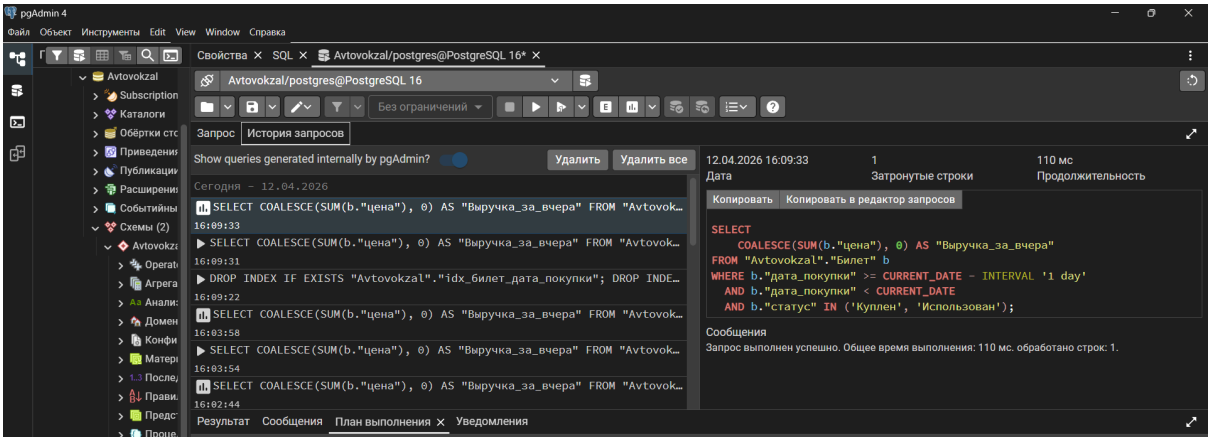
Время выполнения:



Вариант	Время выполнения
Без индекса	61 ms
С индексом	52 ms

1.4.3 История запросов Query History

В процессе выполнения лабораторной работы использовалась вкладка Query History в Query Tool pgAdmin 4. Она позволяет просматривать историю выполненных запросов, включая текст SQL-команды, время выполнения, количество возвращенных строк и служебные сообщения сервера



Выводы

В ходе лабораторной работы были выполнены запросы на выборку данных и созданы представления для базы данных “Vokzal”. Также были составлены запросы на модификацию данных INSERT, UPDATE и DELETE с использованием подзапросов. Для двух запросов были построены планы выполнения с помощью EXPLAIN, созданы простой и составной индексы и выполнено сравнение времени выполнения запросов до и после индексации. В результате были получены практические навыки работы с Query Tool в pgAdmin 4, анализа планов выполнения запросов и использования индексов для повышения производительности.